

Challenge IMA205 - Classification des Globules Blancs

Karen EL KHOURY
Télécom Paris
karen.elkhoury@telecom-paris.fr

Abstract

NOTE: J'ai structuré mon travail en deux notebooks distincts : le premier dédié à une approche basée sur des handcrafted features, et le second à une approche deep learning avec des réseaux de neurones convolutionnels (CNN). Dans ce projet, j'ai étudié différentes approches pour la classification d'images de globules blancs dans un contexte médical, avec un fort déséquilibre entre les classes.

J'ai d'abord exploré une approche basée sur des features faites à la main (morphologie, couleur, texture) combinées à des modèles de machine learning classiques, qui a permis d'obtenir un score F1 macro d'environ 0.56. Cependant, cette approche présente des limitations liées à la segmentation et à la perte d'information.

J'ai ensuite développé une approche deep learning basée sur des réseaux de neurones convolutionnels pré-entraînés (EfficientNet et ResNet). Plusieurs stratégies ont été testées, incluant la gestion du déséquilibre, le fine-tuning, la validation croisée, ainsi que des techniques comme le mixup, le TTA et Hybrid Models (EfficientNet + ResNet).

Le meilleur résultat a été obtenu avec EfficientNet-B0 combiné à une validation croisée stratifiée (5 folds) et un WeightedRandomSampler, atteignant un score F1 macro de 0.7757 sur Kaggle.

1 Introduction

Les globules blancs jouent un rôle central dans le système immunitaire. Leur morphologie (taille, forme du noyau, texture du cytoplasme) permet aux hématologues de diagnostiquer des maladies graves comme la leucémie, les infections bactériennes, virales, ou les anémies.

L'objectif est de classifier automatiquement des images de globules blancs parmi 13 catégories distinctes, en utilisant des méthodes de machine learning et de deep learning.

majoritaires tandis que certaines classes comme PMY ou PLY sont très rares.

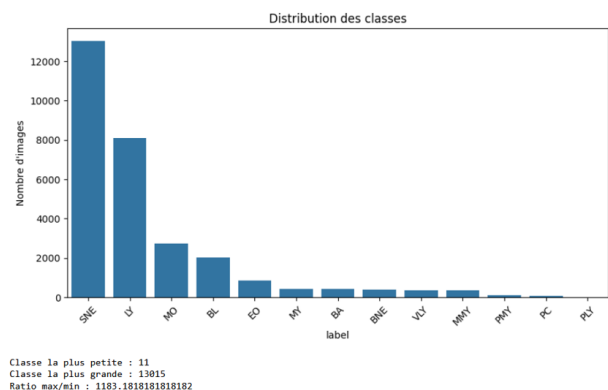


Figure 1: Distribution of classes

2 Données et prétraitement

2.1 Description du dataset

Le dataset contient 28 901 images de globules blancs issues de frottis sanguins colorés au microscope.

- Ensemble train+validation : 70% des données (environ 20 230 images)
- Ensemble test : 30% des données (environ 8 671 images)

J'ai ensuite re-découpé l'ensemble d'entraînement en 80% train et 20% validation avec un split stratifié afin de préserver les proportions des classes. (Dans une autre étape on verra comment j'ai decouper en 5 folds)

2.2 Déséquilibre des classes

Comme le montre la Figure 1 Les classes sont très inégalement représentées. Les neutrophiles (SNE) sont

2.3 Qualité et cohérence des images

Les images présentent une variabilité importante en termes de taille, de résolution et de conditions d'acquisition. Afin de garantir une cohérence dans le pipeline, j'ai redimensionné toutes les images à une résolution fixe de 256×256 pixels pour les étapes de segmentation, et de 64×64 pour les modèles baselines.

Aucune donnée manquante n'a été observée : toutes les images ont pu être chargées sans erreur. Cependant, certaines images présentent des artefacts de coloration ou un bruit de fond important, ce qui rend la segmentation plus délicate dans certains cas. J'ai choisi de

conserver ces images afin de préserver la diversité et le réalisme du dataset.

Les valeurs de pixels ont été normalisées dans l'intervalle $[0,1]$ afin de stabiliser les calculs.

3 Hand-Crafted Feature Extraction

3.1 Segmentation des cellules

Avant d'extraire des features descriptives, il est indispensable de segmenter les images afin d'isoler précisément le noyau et le cytoplasme. Les features morphologiques, colorées et texturales doivent être associées à des régions biologiquement significatives, et non à l'image entière. J'ai donc développé un pipeline de segmentation robuste. J'ai choisi de me concentrer dans un premier temps sur la segmentation du noyau, car il s'agit de la structure la plus facile à isoler dans les images. En effet, le noyau est généralement plus sombre et présente un contraste plus marqué avec le fond. Cette étape constitue une base fiable pour la suite du pipeline. Une fois le noyau correctement segmenté, j'ai ensuite étendu l'analyse au cytoplasme en définissant une région d'intérêt autour du noyau. Cette approche progressive m'a permis de construire une segmentation plus stable et cohérente.

3.1.1 Approche 1 : Seuillage d'Otsu

J'ai commencé par une méthode simple basée sur un seuillage d'Otsu après conversion en niveaux de gris. Cette méthode détermine automatiquement un seuil optimal en maximisant la variance inter-classes.

Un prétraitement CLAHE a été appliqué pour améliorer le contraste local. Cette approche est rapide et simple, mais présente plusieurs limitations :

- Sensibilité aux variations d'illumination
- Mauvaise séparation noyau/fond pour les images à faible contraste
- Résultats instables entre images

Elle ne s'est donc pas révélée suffisamment robuste.

3.1.2 Approche 2 : KMeans

J'ai ensuite exploré une segmentation basée sur le clustering KMeans appliqué aux pixels RGB. L'idée est de regrouper les pixels selon leur couleur et de sélectionner le cluster le plus sombre comme noyau.

Avec $K = 4$, les résultats sont meilleurs que Otsu dans certains cas, mais restent limités :

- Sensibilité à la variation de coloration
- Mauvaise sélection du cluster pour les images difficiles
- Échecs sur certaines classes

Un fallback vers Otsu a été ajouté lorsque le masque était trop petit, mais la méthode reste instable. On voit les résultats dans la Figure 2: Pour quelques images j'arrive à capturer le noyau, mais dans d'autres images le résultat est catastrophique.

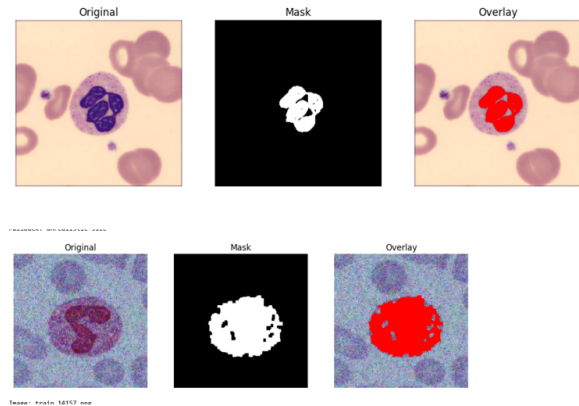


Figure 2: Distribution of classes

3.1.3 Approche finale : Multi-Otsu hiérarchique

J'ai finalement développé un pipeline basé sur Multi-Otsu, plus robuste et cohérent biologiquement.

Description du pipeline Le pipeline se déroule en plusieurs étapes :

- **Étape 1 :** Segmentation du noyau via Multi-Otsu (3 classes). Cette méthode permet de partitionner automatiquement l'image en régions d'intensité. La classe la plus sombre est associée au noyau, ce qui est cohérent avec sa coloration en microscopie.
- **Étape 2 :** Nettoyage morphologique (ouverture, fermeture). Ces opérations permettent de supprimer le bruit, de lisser les contours et de combler les petites imperfections dans le masque. Seul le plus grand composant connexe est conservé afin de garantir la cohérence biologique.
- **Étape 3 :** Définition d'une région d'intérêt (ROI) par dilatation du noyau. Cette étape permet d'englober la cellule complète tout en excluant le fond inutile, ce qui simplifie les étapes suivantes.
- **Étape 4 :** Segmentation interne de la ROI avec Multi-Otsu. Une seconde segmentation est appliquée dans cette région afin de distinguer différentes zones d'intensité, notamment le noyau et le cytoplasme.
- **Étape 5 :** Filtrage des régions connectées au noyau. Cette contrainte permet de conserver uniquement les structures appartenant réellement à la cellule, en éliminant les artefacts ou objets parasites.

- **Étape 6 : Raffinement final.** Des opérations morphologiques supplémentaires sont appliquées pour améliorer la qualité des masques. Le masque final de la cellule est extrait, et le cytoplasme est obtenu par différence avec le noyau.

La Figure 3 illustre le pipeline complet de segmentation sur deux exemples d'images. Chaque ligne correspond à une image différente, et chaque colonne représente une étape clé du processus. La première colonne montre l'image originale. La deuxième colonne correspond au seuillage global Multi-Otsu en 3 classes, permettant de séparer les régions selon leur intensité. La troisième colonne montre le masque du noyau après nettoyage morphologique. La quatrième colonne représente la région d'intérêt (ROI), obtenue par dilatation du noyau afin d'inclure la cellule entière.

La cinquième colonne montre le seuillage Multi-Otsu appliqué uniquement dans la ROI, permettant de mieux distinguer les différentes structures internes. La sixième colonne correspond aux régions candidates pour la cellule, après filtrage des composantes connexes. La septième colonne montre le masque final de la cellule après raffinement. Enfin, la dernière colonne présente une superposition (overlay) du masque final sur l'image originale, avec le noyau et le cytoplasme clairement identifiés.

Ces résultats montrent que le pipeline permet d'isoler efficacement le noyau puis la cellule entière, même en présence de bruit ou de variations d'intensité.

3.1.4 Avantages

Cette approche présente plusieurs avantages :

- Adaptation automatique via Multi-Otsu (pas de seuil fixe)
- Cohérence biologique (noyau → cytoplasme)
- Robustesse au bruit grâce aux opérations morphologiques
- Exclusion des artefacts via la connectivité

Malgré ces améliorations, certaines images très bruitées ou présentant des chevauchements restent difficiles à segmenter. Ces cas restent minoritaires et n'affectent pas la robustesse globale du pipeline.

Extraction de features

Après avoir mis en place un pipeline de segmentation fiable, j'ai extrait des features descriptives à partir des masques du noyau, de la cellule et du cytoplasme. L'objectif est de transformer chaque image en un vecteur numérique de 83 dimensions, contenant des informations morphologiques, colorées et texturales.

3.2 Features morphologiques

Les features morphologiques décrivent la géométrie du noyau et de la cellule.

Le ratio N/C est particulièrement important en hématologie, car il distingue les cellules matures des cellules immatures.

Feature	Région	Description
Aire	Noyau/Cellule	Nombre de pixels
Périmètre	Noyau/Cellule	Longueur du contour
Circularité	Noyau/Cellule	$4\pi A/P^2$
Grand axe	Noyau/Cellule	Axe principal
Petit axe	Noyau/Cellule	Axe secondaire
Aspect ratio	Noyau/Cellule	Rapport axes
Solidité	Noyau/Cellule	Aire / aire convexe
Extent	Noyau/Cellule	Aire / bounding box
Excentricité	Noyau/Cellule	Forme elliptique
Convexité	Noyau/Cellule	Périmètre convexe / réel
Ratio N/C	Noyau/Cellule	Aire noyau / cellule

Table 1: Features morphologiques

3.3 Features de couleur

3.3.1 Couleur RGB

Pour chaque région (noyau et cytoplasme), j'ai extrait :

- Moyenne des canaux R, G, B
- Écart-type des canaux R, G, B
- Skewness des distributions

Ces statistiques capturent la couleur dominante et la variabilité interne.

3.3.2 Espaces HSV et LAB

Pour améliorer la représentation, j'ai utilisé :

- HSV : teinte, saturation, luminosité
- LAB : luminance (L), chrominance (a, b)

Pour chaque espace et région, j'ai calculé la moyenne et l'écart-type des canaux.

3.4 Features de texture

J'ai utilisé la GLCM (Gray Level Co-occurrence Matrix) pour capturer la texture :

Descripteur	Signification
Contraste	Variation locale d'intensité
Homogénéité	Uniformité de la texture
Énergie	Régularité
Corrélation	Structure des pixels

Table 2: Descripteurs GLCM

La GLCM est calculée sur plusieurs distances et angles, puis moyennée pour obtenir des descripteurs invariants.

Résumé des features Ce vecteur de 83 features constitue une représentation compacte et interprétable des cellules, adaptée aux modèles de machine learning classiques.

Catégorie	Nb features
Morphologie	23
Couleur RGB	18
HSV + LAB	24
Texture GLCM	18
Total	83

Table 3: Répartition des features

3.5 Analyse et visualisation des features

3.5.1 Normalisation des features

Avant l'entraînement, j'ai normalisé les features à l'aide d'un `StandardScaler`. Cette étape est essentielle car les features ont des échelles très différentes (par exemple, l'aire peut être de l'ordre de milliers tandis que les features GLCM sont inférieures à 1).

Le `StandardScaler` centre les données et les réduit à variance unitaire. Il est ajusté uniquement sur les données d'entraînement puis appliqué à la validation et au test afin d'éviter toute fuite d'information.

3.5.2 Visualisation par PCA

Pour analyser la structure des données et évaluer la séparabilité des classes, j'ai appliqué une Analyse en Composantes Principales (PCA) afin de projeter les 83 features dans un espace de dimension 2.

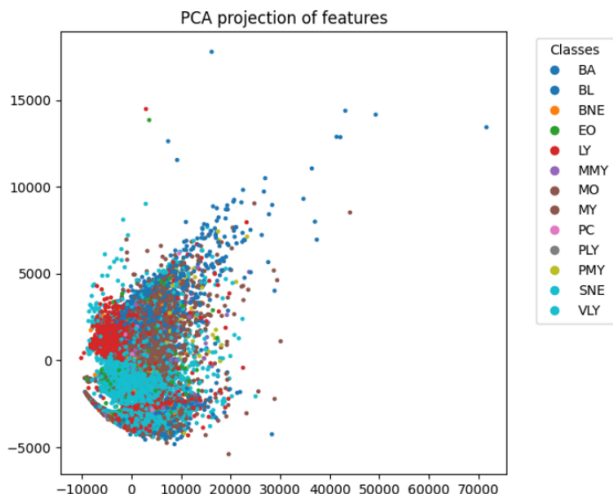


Figure 4: Projection PCA des 83 features en 2 dimensions. Chaque point correspond à une cellule, colorée selon sa classe. On observe un fort chevauchement entre les classes.

La Figure 4 montre la projection obtenue. On observe un fort chevauchement entre les différentes classes, sans formation de clusters bien séparés. Les points appartenant à des classes distinctes occupent globalement les mêmes régions de l'espace, ce qui indique que les frontières entre classes sont complexes et fortement non linéaires.

Certaines classes majoritaires forment des regroupements denses, tandis que les classes rares sont dispersées et difficilement distinguables. De plus, la forte superposition entre certaines classes biologiquement proches (par exemple les précurseurs myéloïdes) confirme que les différences entre ces cellules reposent sur des variations fines, difficiles à capturer dans un espace de faible dimension.

Ces observations montrent que les features extraites, bien qu'informatives, ne permettent pas une séparation linéaire des classes. Cela justifie l'utilisation de modèles capables de capturer des relations non linéaires complexes, comme les SVM à noyau RBF ou les modèles d'ensemble basés sur des arbres (Random Forest, XGBoost).

3.6 Modèles de classification

Après extraction et normalisation des features, j'ai entraîné plusieurs modèles de machine learning classiques afin d'évaluer la capacité de ces descripteurs à discriminer les différentes classes. L'évaluation est réalisée à l'aide du macro F1-score, plus adapté que l'accuracy dans un contexte de fort déséquilibre.

3.6.1 Random Forest

Modèle de base J'ai commencé par un Random Forest avec 600 arbres et `class_weight='balanced'`. Ce modèle présente plusieurs avantages : il est robuste, non linéaire et capable de capturer des interactions complexes entre les features.

Les résultats montrent une accuracy élevée (~ 0.86) mais un macro F1-score faible (~ 0.39). Cette différence s'explique directement par le déséquilibre des classes : le modèle apprend efficacement les classes majoritaires mais échoue à prédire correctement les classes rares.

Cela met en évidence une limite importante : même avec un rééquilibrage des poids, le modèle reste biaisé vers les classes dominantes.

Balanced Random Forest Pour corriger ce problème, j'ai testé une version avec sous-échantillonnage automatique des classes majoritaires.

Cependant, les performances diminuent (~ 0.35). Ce résultat s'explique par une perte d'information : en réduisant artificiellement les classes majoritaires, le modèle perd des exemples utiles et introduit plus de variance dans l'apprentissage.

Cela montre qu'un rééquilibrage trop agressif peut être contre-productif.

3.6.2 XGBoost

J'ai ensuite utilisé XGBoost, un modèle de gradient boosting qui construit les arbres de manière séquentielle en corrigeant les erreurs précédentes.

Configuration utilisée :

- 800 estimateurs

- profondeur maximale = 6
- learning rate = 0.05
- subsample = 0.8

XGBoost obtient un F1 macro d'environ 0.45, ce qui représente une amélioration notable par rapport au Random Forest.

Cette amélioration s'explique par :

- une meilleure modélisation des interactions complexes
- une optimisation progressive des erreurs
- une meilleure gestion du compromis biais/variance

Cependant, le modèle reste limité sur les classes rares, ce qui montre que les features extraites ne permettent pas une séparation parfaite.

3.6.3 SVM à noyau RBF

Le modèle qui donne les meilleures performances est le SVM avec noyau RBF.

Contrairement aux modèles précédents, le SVM ne construit pas explicitement des arbres mais cherche une frontière de décision optimale dans un espace transformé.

Le noyau RBF projette les données dans un espace de dimension élevée, permettant de capturer des relations fortement non linéaires.

Cette approche est particulièrement adaptée ici car :

- les classes ne sont pas linéairement séparables (confirmé par la PCA)
- les features sont de dimension relativement élevée (83)
- le dataset reste de taille modérée

Le SVM atteint un F1 macro d'environ 0.50 sans optimisation.

3.6.4 Optimisation des hyperparamètres

Pour améliorer les performances, j'ai effectué une recherche d'hyperparamètres avec GridSearchCV (3 folds) sur :

- $C \in \{1, 5, 10\}$
- $\gamma \in \{scale, 0.1, 0.01\}$

Le paramètre C contrôle le compromis entre marge et erreurs, tandis que γ détermine la portée du noyau RBF.

Les meilleurs résultats sont obtenus avec :

$$C = 5, \quad \gamma = scale$$

Ce choix correspond à un bon équilibre entre sous-apprentissage et sur-apprentissage.

Le modèle optimisé atteint un F1 macro d'environ 0.56.

3.7 Discussion

Ces résultats mettent en évidence plusieurs points importants :

- Les modèles linéaires sont insuffisants pour ce problème
- Les méthodes basées sur les arbres améliorent les performances mais restent limitées
- Le SVM RBF est le plus adapté aux données non linéaires
- La qualité des features est un facteur limitant

3.8 Limites de l'approche

L'approche basée sur les handcrafted features présente plusieurs limitations fondamentales.

Dépendance à la segmentation : Toutes les features extraites reposent sur la qualité de la segmentation. Une erreur dans le masque du noyau ou du cytoplasme se propage directement dans les features, ce qui peut conduire à des prédictions incorrectes. Cet effet de cascade est difficile à corriger a posteriori.

Perte d'information : La représentation d'une image par un vecteur de 83 valeurs scalaires implique une perte d'information. Des structures spatiales fines, des textures complexes ou des configurations morphologiques spécifiques peuvent ne pas être correctement capturées par des descripteurs statistiques.

Sensibilité aux artefacts : Les features de couleur sont sensibles aux variations de coloration entre les lames, tandis que les features de texture peuvent être affectées par le bruit d'acquisition. Cela limite la capacité de généralisation du modèle sur de nouvelles données.

3.9 Transition vers le deep learning

Ces limitations montrent que l'approche par features manuelles atteint rapidement ses limites pour un problème aussi complexe et non linéaire.

Cela motive naturellement l'utilisation de méthodes de deep learning, et en particulier des réseaux de neurones convolutionnels (CNN), qui permettent d'apprendre directement des représentations pertinentes à partir des images brutes.

4 Approche Deep Learning

Suite aux limites observées avec les handcrafted features, j'ai exploré une approche basée sur le deep learning en utilisant des réseaux de neurones convolutionnels (CNN).

Contrairement à l'approche précédente, les CNN permettent d'apprendre automatiquement des représentations à partir des images brutes, sans nécessiter d'étapes explicites de segmentation ou d'extraction de features. Cela permet notamment de

capturer des patterns visuels complexes, difficiles à modéliser manuellement.

Dans ce travail, j'ai utilisé des modèles pré-entraînés sur ImageNet (transfer learning), car mon dataset est de taille limitée et ne permet pas d'entraîner un réseau profond from scratch de manière stable.

4.1 Pipeline de prétraitement

Étape	Description
Données brutes	28 901 images, split 70/30
Prétraitement	Resize 224×224, normalisation ImageNet, split stratifié
Augmentation	Crop, flips, rotation, ColorJitter
Dataset PyTorch	Classe personnalisée WBC-Dataset
Imbalance	Class weights puis sampler
Architecture	EfficientNet-B0 / ResNet18 (ImageNet)
Entraînement	Adam, lr=1e-4, dropout 0.3, 40 epochs
Évaluation	Macro F1-score
Améliorations	5-folds, mixup, TTA, ensemble

Table 4: Pipeline complet du projet

4.2 Transformations et augmentation de données

Pour l'entraînement, j'ai appliqué les transformations suivantes :

- RandomResizedCrop(224)
- RandomHorizontalFlip et RandomVerticalFlip
- RandomRotation(30°)
- ColorJitter
- Normalisation ImageNet

Pour la validation et le test, seules les transformations suivantes sont utilisées :

- Resize 224×224
- Normalisation

J'ai initialement testé des augmentations plus agressives, mais cela rendait l'entraînement instable. J'ai donc réduit leur intensité pour obtenir un meilleur compromis.

4.3 Plan des expérimentations

Afin d'optimiser les performances, j'ai exploré plusieurs axes :

- **Choix de l'architecture** : comparaison entre ResNet18 et EfficientNet

- **Stratégies de fine-tuning** : freezing du backbone vs entraînement complet
- **Gestion du déséquilibre** : class weights vs WeightedRandomSampler
- **Régularisation** : dropout et data augmentation
- **Validation** : utilisation de la cross-validation (5 folds)
- **Améliorations** : mixup et Test Time Augmentation (TTA)
- **Ensemble** : combinaison de plusieurs modèles (hybrid)

Chaque expérimentation sera analysée en détail, en expliquant les résultats obtenus et leur interprétation théorique.

5 Architectures CNN utilisées

5.1 Choix du Transfer Learning

J'ai utilisé des modèles pré-entraînés sur ImageNet plutôt que d'entraîner un CNN from scratch.

Cette décision repose sur plusieurs raisons :

- Le dataset est insuffisant pour entraîner un réseau profond de manière stable
- Les modèles ImageNet apprennent déjà des features bas-niveau utiles
- Le fine-tuning permet d'adapter ces features aux images médicales

De plus, le challenge impose l'utilisation de modèles pré-entraînés sur ImageNet uniquement.

5.2 EfficientNet-B0

EfficientNet-B0 est le modèle principal utilisé. Il repose sur un scaling conjoint de la profondeur, largeur et résolution.

Architecture adaptée :

Couche	Détail
Input	224 × 224 × 3
Dropout	0.3
FC	Linear(1280, 13)
Sortie	Softmax

Table 5: Architecture EfficientNet adaptée

La dernière couche a été remplacée pour adapter le modèle aux 13 classes.

5.3 Stratégie d'entraînement

5.3.1 Optimisation

J'ai utilisé Adam avec :

- LR = 10^{-4} (EfficientNet), 3×10^{-4} (ResNet)
- Weight decay = 10^{-4}
- Scheduler : ReduceLROnPlateau

Un LR trop élevé rendait l'entraînement instable.

5.3.2 Fonction de perte

J'ai utilisé CrossEntropyLoss.

Deux approches testées :

- Class weights
- WeightedRandomSampler

Résultats :

- Class weights : F1 = 0.67
- Sampler : F1 = 0.70

Le sampler est plus efficace car il agit directement sur les batches, on verra ça en détails plus tard dans le rapport.

5.3.3 Checkpoint

Le meilleur modèle est sauvegardé selon le F1 validation, permettant une reprise et une meilleure sélection finale.

5.4 Expériences et résultats

Dans cette section, je détaille le processus complet qui m'a permis d'améliorer progressivement les performances du modèle. L'objectif n'était pas uniquement de tester différentes techniques, mais de comprendre à chaque étape pourquoi certaines approches fonctionnent et d'autres non.

5.4.1 Point de départ : EfficientNet baseline

J'ai commencé avec EfficientNet-B0 fine-tuné sur le dataset, avec une gestion classique du déséquilibre via les class weights.

Le score obtenu est de 0.67.

Ce résultat constitue une baseline cohérente, mais limitée. Le modèle apprend correctement les classes majoritaires, mais reste pénalisé sur les classes rares. Cela montre que la gestion du déséquilibre est un facteur clé.

5.4.2 Gestion du déséquilibre : sampler vs class weights

J'ai ensuite remplacé les class weights par un WeightedRandomSampler.

Le score passe de 0.67 à 0.70.

Cette amélioration confirme que le sampler est plus efficace dans ce contexte, car il agit directement sur la distribution des données vues par le modèle. Les batches deviennent plus équilibrés, ce qui améliore l'apprentissage des classes rares.

À ce stade, j'ai compris que le problème principal n'était pas seulement l'architecture, mais la manière dont les données sont présentées au modèle.

5.4.3 Exploration des architectures : ResNet vs EfficientNet

J'ai testé ResNet18 comme alternative.

Le score reste autour de 0.67, quelle que soit la configuration.

Ce résultat montre que ResNet18 atteint rapidement un plafond de performance. J'en ai conclu que le problème vient d'un biais structurel : le modèle est trop simple pour capturer les différences fines entre certaines classes.

À l'inverse, EfficientNet atteint 0.70, ce qui confirme sa meilleure capacité de représentation.

5.4.4 Ensemble initial : Hybrid ResNet + EfficientNet

J'ai ensuite combiné ResNet18 et EfficientNet avec une moyenne pondérée (60/40), en ajoutant du TTA.

Le score monte à 0.76.

Cette amélioration est significative. Elle s'explique par le fait que les deux modèles font des erreurs différentes, et leur combinaison réduit la variance.

5.4.5 Test Time Augmentation (TTA)

J'ai ensuite introduit le Test Time Augmentation (TTA) afin d'améliorer la robustesse des prédictions.

Le principe est de faire plusieurs prédictions pour une même image en appliquant différentes transformations simples (par exemple des flips horizontaux), puis de moyenner les probabilités obtenues.

Cette approche permet de réduire la sensibilité du modèle aux variations d'entrée et de lisser les prédictions.

En pratique, l'ajout du TTA dans le modèle hybride (EfficientNet + ResNet18) a contribué à une amélioration significative du score, passant d'environ 0.70 à 0.76.

Cette amélioration s'explique par une réduction de la variance du modèle : en moyennant plusieurs prédictions légèrement différentes, on diminue l'impact des erreurs aléatoires.

Cependant, j'ai également observé que le TTA peut devenir moins utile lorsque le modèle est déjà très optimisé (par exemple avec EfficientNet + 5 folds). Dans

ce cas, il peut introduire un effet de lissage excessif et légèrement dégrader les performances.

Cela montre que, comme pour les ensembles, le TTA est efficace surtout lorsqu'il apporte une diversité pertinente, mais peut devenir inutile voire contre-productif lorsqu'il est combiné à des modèles déjà très robustes.

5.4.6 5-Fold Stratified

Principe La validation croisée à 5 folds consiste à diviser le dataset en 5 parties égales. À chaque itération, une partie est utilisée pour la validation et les 4 autres pour l'entraînement, ce qui permet d'entraîner 5 modèles différents.

	P1	P2	P3	P4	P5
Fold 1	Val	Train	Train	Train	Train
Fold 2	Train	Val	Train	Train	Train
Fold 3	Train	Train	Val	Train	Train
Fold 4	Train	Train	Train	Val	Train
Fold 5	Train	Train	Train	Train	Val

Table 6: Schéma de la validation croisée

J'ai utilisé **StratifiedKFold** afin de préserver la distribution des classes dans chaque fold.

Avantages L'application des folds à EfficientNet-B0 a permis une amélioration significative ($0.70 \rightarrow 0.7757$).

Cela s'explique par :

- **Réduction de la variance** : les performances ne dépendent plus d'un seul split
- **Meilleure généralisation** : le modèle voit plus de données
- **Effet ensemble** : les modèles font des erreurs différentes

Les prédictions finales sont obtenues en moyennant les sorties des 5 modèles.

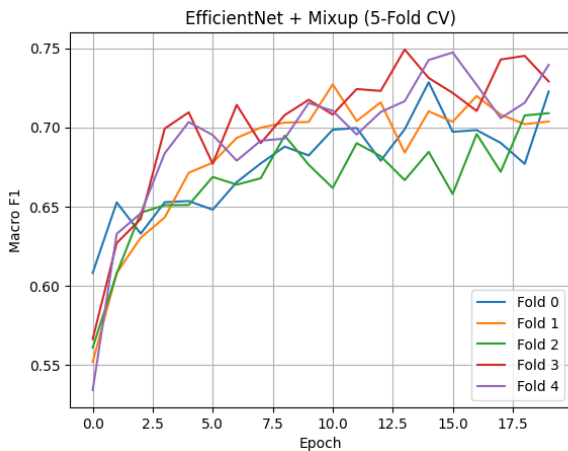


Figure 5: Évolution du score F1 macro au cours des epochs pour les 5 folds avec EfficientNet et mixup. Chaque courbe correspond à un fold différent.

Comme le montre la Figure 5, les performances (F1 macro) augmentent globalement au fil des epochs pour les différents folds.

On observe cependant une certaine variabilité entre les folds, ce qui reflète la sensibilité du modèle aux différentes partitions des données. Malgré cette variabilité, les courbes restent cohérentes et convergent vers des performances similaires, indiquant un apprentissage stable.

Les fluctuations observées peuvent s'expliquer par la présence du mixup, qui introduit une forme de régularisation en rendant les exemples plus difficiles à apprendre. Cela peut ralentir la convergence mais améliore la robustesse globale du modèle.

5.4.7 Application à ResNet18

Avec ResNet18, les folds n'ont pas amélioré les performances ($F1 \sim 0.67$).

Cela montre que le problème est lié au biais du modèle, et non à la variance.

Conclusion : les folds réduisent la variance mais ne corrigent pas un modèle limité.

5.4.8 Mixup et régularisation avancée

Principe du Mixup Le Mixup consiste à générer des exemples interpolés entre deux images et leurs labels :

$$x' = \lambda x_i + (1 - \lambda)x_j$$

$$L = \lambda L(y_i) + (1 - \lambda)L(y_j)$$

Cette technique agit comme une régularisation en forçant le modèle à apprendre des transitions plus lisses entre les classes, ce qui permet en théorie d'améliorer la généralisation et de réduire le surapprentissage.

Résultats et analyse Dans mon cas, EfficientNet avec mixup atteint un score de 0.7525, ce qui est inférieur au modèle avec 5 folds seul (0.7757).

Ce résultat peut s'expliquer par la nature des données. Contrairement à des images naturelles, les cellules sanguines présentent des structures morphologiques précises et discrètes. En appliquant le mixup, on génère des images interpolées qui ne correspondent pas à des configurations biologiquement réalistes.

Ainsi, au lieu d'aider le modèle, le mixup introduit des exemples artificiels qui perturbent l'apprentissage. Le modèle doit alors apprendre à gérer des distributions non naturelles, ce qui dégrade légèrement les performances.

Pour ResNet18, les performances chutent encore davantage (0.6463). Cela s'explique par la combinaison de deux effets :

- un biais élevé du modèle (capacité limitée de ResNet18)

- une complexification des données induite par le mixup

Le modèle n'a pas suffisamment de capacité pour apprendre efficacement à partir de ces données plus difficiles.

5.4.9 Hybrid Models

J'ai ensuite exploré plusieurs combinaisons de modèles afin d'évaluer l'impact de la diversité introduite par le mixup :

- Hybrid Mixup (EffNet + ResNet) : 0.7734
- EfficientNet folds + Mixup : 0.7734
- EfficientNet folds + TTA : 0.7576

Ces résultats mettent en évidence un point important : un ensemble de modèles n'est efficace que si chaque modèle apporte une contribution utile.

Même si le mixup introduit de la diversité, les modèles entraînés avec cette technique sont légèrement moins performants individuellement. En les combinant avec des modèles plus forts (comme EfficientNet avec 5 folds), ils tirent la moyenne vers le bas.

De plus, l'ajout du TTA introduit un effet de lissage supplémentaire. Lorsqu'il est combiné avec des modèles déjà optimisés, cela peut réduire la confiance des prédictions et dégrader la précision finale.

6 Conclusion

Ce travail m'a permis d'explorer différentes approches pour la classification d'images de globules blancs, allant des features handcrafted aux méthodes de deep learning.

6.1 Score final

Modèle	F1 Kaggle
EfficientNet-B0 + 5 folds + sampler	0.7757

Table 7: Meilleur résultat

Ce score constitue un résultat solide compte tenu des contraintes du challenge.

La stratégie la plus efficace a été de privilégier une architecture performante et d'exploiter au maximum les données via la validation croisée, plutôt que de multiplier les modèles complexes. En conclusion, ce travail montre que dans un contexte de données limitées et déséquilibrées, la simplicité et la rigueur dans l'expérimentation sont souvent plus efficaces que la complexité.

6.2 Synthèse des résultats

Les expérimentations ont montré une amélioration progressive des performances au fil des différentes techniques testées. Comme le montrent la Figure 6 et la Table 8, le gain le plus significatif provient de l'utilisation de la validation croisée (5 folds).

Le meilleur modèle obtenu est EfficientNet-B0 combiné à un WeightedRandomSampler et une validation croisée, atteignant un score F1 macro de 0.7757.

6.3 Discussion

Les résultats mettent en évidence un point important : l'ajout de complexité ne garantit pas une amélioration des performances.

Un ensemble de modèles est efficace uniquement si :

- les modèles sont suffisamment performants individuellement
- ils apportent une diversité réellement complémentaire

Dans mon cas, EfficientNet avec 5 folds est déjà fortement optimisé. L'ajout de modèles plus faibles (comme ResNet18) ou de techniques supplémentaires (mixup, TTA) n'apporte pas de gain, et peut même introduire du bruit en lissant excessivement les prédictions.

6.4 Enseignements principaux

Les principaux enseignements de ce travail sont les suivants :

- La gestion du déséquilibre des classes est essentielle (le sampler est plus efficace que les class weights)
- Le choix de l'architecture est déterminant : EfficientNet surpasse clairement ResNet18
- La validation croisée (5 folds) est la technique la plus efficace pour améliorer les performances
- Les méthodes avancées (mixup, ensembles complexes) ne garantissent pas une amélioration
- La compréhension du dataset est plus importante que l'ajout de techniques complexes

6.5 Perspectives

Plusieurs pistes d'amélioration peuvent être envisagées :

- Tester des architectures plus avancées (EfficientNet-B4, Vision Transformers)
- Combiner les approches CNN et les features handcrafted
- Explorer des fonctions de perte adaptées au déséquilibre (ex : focal loss)

Expérience	Description	F1
EffNet + weights	EfficientNet + class weights	0.67
EffNet + sampler	EfficientNet + sampler	0.70
ResNet18	Baseline	0.67
EffNet	Fine-tuning	0.70
Hybrid	EffNet + ResNet + TTA	0.76
ResNet folds	5 folds	0.67
EffNet folds	5 folds	0.7757
Hybrid folds	Ensemble folds	0.77
EffNet mixup	Mixup	0.7525
ResNet mixup	Mixup	0.6463
Hybrid mixup	Ensemble mixup	0.7734
Hybrid EffNet	folds + mixup	0.7734
EffNet + TTA	folds + TTA	0.7576

Table 8: Résumé des performances des différentes expérimentations CNN

6.6 Conclusion générale

Finalement, la meilleure stratégie a été de privilégier une approche simple mais robuste : EfficientNet-B0 avec 5 folds et un sampler.

Ce travail montre que, dans un contexte de données déséquilibrées et limitées, l’optimisation des fondamentaux (données, architecture, validation) est plus efficace que l’ajout de techniques complexes.

[7] Z. Kastrati et al., *Deep learning for classification of medical images*, Information Processing and Management, 2020.

6.7 Pistes d’amélioration

- Essayer d’autres CNN Pre-entraîner
- Combiner CNN et features handcrafted
- Utiliser des fonctions de perte adaptées (ex : focal loss)

References

- [1] S. S. Banik et al., *An automatic nucleus segmentation and CNN model based classification method of white blood cells*, Expert Systems with Applications, 2020.
- [2] S. Acevedo et al., *Recognition of peripheral blood cell images using convolutional neural networks*, Computer Methods and Programs in Biomedicine, 2019.
- [3] A. Asghar et al., *Classification of white blood cells using machine learning and deep learning models*, IEEE Access, 2020.
- [4] H. Danyali et al., *Robust leukocyte segmentation in blood microscopic images*, Biomedical Engineering Conference, IEEE, 2015.
- [5] S. Safuan et al., *White blood cell counting analysis using various color segmentation methods*, Measurement Journal, 2018.
- [6] X. Wang et al., *Few-shot leukocyte classification using EfficientNet-based models*, MDPI Healthcare, 2022.

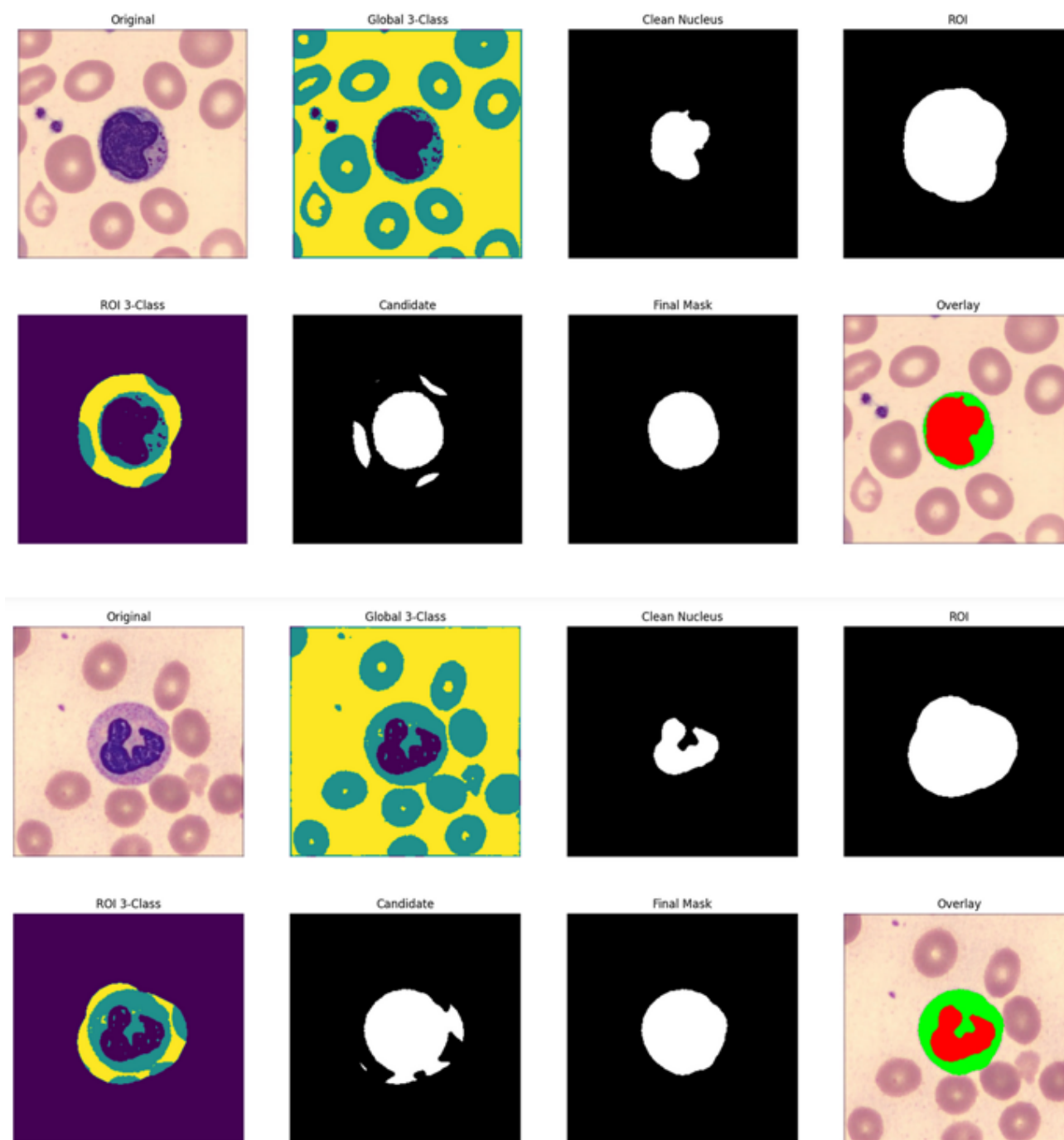


Figure 3: Pipeline de segmentation des cellules. Chaque ligne correspond à une image différente. De gauche à droite : image originale, segmentation globale Multi-Otsu (3 classes), noyau nettoyé, région d'intérêt (ROI), segmentation interne de la ROI, régions candidates, masque final, et superposition du résultat sur l'image originale.

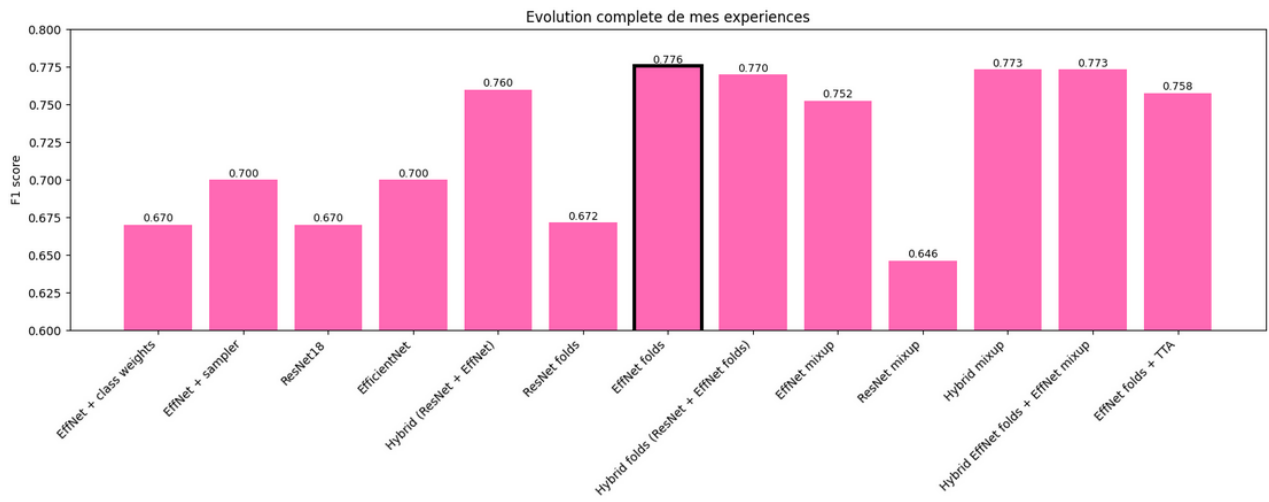


Figure 6: Évolution complète des performances des différents modèles testés. On observe que la meilleure performance est obtenue avec EfficientNet-B0 en validation croisée (5 folds), atteignant un F1 macro de 0.7757.